

Experiments Report

Enterprise Document Intelligence Platform

Maryam Fatima

Track: NLP and AI Agents

This report documents four experiments performed on the actual Enterprise Document Intelligence Platform, using “The Last Leaf” by O. Henry (a public-domain short story, 10,863 characters) as the test document — chosen deliberately as content unrelated to the app's own subject matter, to confirm the pipeline works correctly on any text, not just technical documents.

Experiment 1: Chunk Size

The document was split using chunk sizes of 300, 500, and 1000 characters, with overlap held constant at 100.

Chunk Size	Number of Chunks	Average Chunk Length
300	55	295 characters
500	28	483 characters
1000	13	925 characters

Discussion

The pattern matches the earlier test on a different document exactly: as chunk size increased, the number of chunks fell sharply (55 → 28 → 13), since the same 10,863-character story needs more small pieces or fewer large ones to cover it fully. This consistency across two very different documents (a technical journal versus a narrative short story) is itself a useful finding: the chunking behavior is predictable and driven purely by document length and the chosen size, regardless of writing style or subject matter.

Experiment 2: Chunk Overlap

The document was split with chunk size fixed at 500, while overlap was varied across 0, 100, and 200 characters.

Overlap	Number of Chunks
0	22
100	28
200	37

Discussion

As overlap increased, so did the chunk count (22 → 28 → 37), for the same reason as before: more shared text between neighboring chunks means more chunks are needed to cover the same document. The real boundary text again shows exactly why overlap matters. With overlap = 0:

End of chunk 1: “...ney. Suppose a man came to get the money. The man might wal”

Start of chunk 2: “k down that street and suddenly meet himself coming back...”

The word “walk” is split directly across the chunk boundary. With overlap = 100 or 200, chunk 2 instead begins by repeating earlier text from chunk 1 (“Suppose a man ca...” / “Suppose a painter...”), keeping the surrounding sentences intact in at least one chunk. This confirms the same conclusion as the first test document: overlap directly trades extra storage for fewer broken sentences.

Experiment 3: Prompt Templates

The same question and retrieved context were sent to Gemini using three prompt wordings. (Note: on the first run, loose_no_instruction hit a temporary Gemini 503 overload; simply rerunning the script succeeded, confirming the error was transient and not a bug — itself a small useful data point about API reliability.)

strict_grounding

“This document is about ‘The Last Leaf’ by O. Henry. It describes Greenwich Village and introduces two characters, Sue and Johnsy.”

loose_no_instruction

A longer, multi-part answer: names the story and author, describes Greenwich Village as “a unique neighborhood... popular, affordable home for artists,” introduces “two young women, Sue and Johnsy,” and adds an inferred detail not explicitly asked for: “a hint of a developing plot as a ‘cold stranger’ (likely an illness) enters the village as winter approaches.”

citation_required

“The document is about ‘The Last Leaf’ by O. Henry, which is a story set in Greenwich Village that introduces characters like Sue and Johnsy,” citing only the author and title lines as sources this time.

Discussion

With all three prompts now successful, the length and behavior differences are clear and consistent with the earlier test document: strict_grounding gave the shortest, most tightly scoped answer; loose_no_instruction gave the longest and most elaborate one, and even added an inference (“likely an illness”) that wasn't explicitly stated in the prompt's instructions — useful detail, but a small reminder that looser prompts give the model more room to interpret rather than strictly report.

One additional finding worth noting: citation_required's citation list was noticeably thinner on this run (just the author and title lines) than in the earlier test on a different document, where it cited full descriptive sentences. Same prompt template, same model, different result — a small but real example of how LLM outputs can vary run to run even when nothing in the setup changes, which is itself a relevant consideration when designing a system that depends on consistent citation behavior.

Experiment 4: Embedding Models

The same chunk of text was embedded using Gemini's cloud-based model and a free, local model from the sentence-transformers library.

Model	Type	Vector Length
gemini-embedding-001	Cloud API (Google Gemini)	3072
all-MiniLM-L6-v2	Local (sentence-transformers)	384

Discussion

Results were identical in kind to the first test: Gemini produced a much larger vector (3072 numbers) than the local MiniLM model (384 numbers). This is consistent and expected regardless of the document's content — vector length is a property of the model itself, not of the text being embedded. Gemini remains the stronger choice while API access is available and reliable; the local model is a free, offline-capable fallback with no rate limits, which is notably relevant given the 503 error observed in Experiment 3 — a local embedding model would not be affected by Gemini's server load at all, though the generation step would still depend on some LLM being available.

Overall Findings

Running these experiments on a second, unrelated document (a short story rather than a technical document) confirmed that the platform's behavior is consistent and driven by the settings chosen, not by the specific content uploaded — chunk size and overlap produced the same patterns as the first test document. Experiment 3 also produced a useful side-

finding: a transient Gemini 503 error on the first attempt, resolved simply by rerunning, which is a real demonstration of why the platform's graceful error handling matters for a production system. Taken together with the first round of experiments, the conclusion holds: every setting in this pipeline — chunk size, overlap, prompt wording, and embedding model — is a deliberate trade-off, and a dependable system must also plan for the external AI provider occasionally being unavailable or inconsistent between runs.